Trabalho 2 – Árvores Binárias e Análise de Complexidade

Grupo: Kaio Henrique da Silva Nezio, Thiago Fabiano e Daniel Pinheiro

Seção 1 - Relato do desenvolvimento das bibliotecas e do programa

Esta seção descreve como o trabalho foi dividido entre os componentes do grupo, onde está o código e como as ferramentas de IA generativa (LLMs) foram usadas.

Observação: as seções 1 a 3 foram entregues na primeira parte do trabalho e foram revisadas para esta versão final. Corrigimos alguns valores e interpretações das medições; ajustamos trechos para ficarem coerentes com a árvore AVL (seções 4 a 6); Os tempos medidos, as listagens de código e as conclusões de complexidade não mudaram.

1.1 Atuação dos componentes do grupo

Parte	Tarefa	Responsável
1	Classe No e esqueleto da ArvoreBinaria, com o construtor que recebe o Comparator	Daniel
1	Métodos adicionar (sem chaves duplicadas), pesquisar e quantidadeNos	Daniel
1	Método remover (folha, um filho e dois filhos, com o sucessor)	Thiago
1	Métodos altura, caminharEmOrdem e caminharEmNivel	Thiago
1	Main com as três opções de estrutura, sem casts para ListaEncadeada	Kaio
1	GeradorArquivosOrdenados, GeradorArquivosBalanceados e a escolha do elemento de pior caso	Kaio
1	Primeira coleta de tempos com os oito arquivos	Kaio
1	Reorganização do programa no padrão MVC (view, controller e service) e Makefile	Thiago
1	Atualização do README e nova coleta de tempos da seção 3	Thiago
1	GeradorNomes (nomes únicos) e correção da remoção na coleção de nomes do ContatoService	Thiago
1	Relatório: seção 1, introdução da seção 2 e seções 2.1, 2.2 e 2.4	Daniel
1	Relatório: seção 2.3 e comparação com as listas (final da 3.2)	Thiago
1	Relatório: seção 3.1, tabelas e interpretação da 3.2	Kaio
2	ArvoreAVL: classe base herdando de ArvoreBinaria e métodos adicionar e remover recursivos	Thiago
2	ArvoreAVL: altura no No, fator de balanceamento, rotações e método balancear	Thiago
2	Opção 4 (Árvore AVL) na ContatoView e na Main	Kaio
2	Medições com a AVL (10 execuções por arquivo), tabelas e gráficos da seção 5	Kaio

Parte	Tarefa	Responsável
2	Relatório: seções 4.1 e 4.2 (altura da AVL e métodos de balanceamento)	Daniel
2	Relatório: seções 4.3 a 4.6 (adicionar e remover da AVL)	Thiago
2	Relatório: seção 5 (implementação, interpretação e comparação com a árvore comum)	Kaio
2	Relatório: seção 6 (TreeMap e TreeSet)	Daniel
2	Atualização do README e revisão final do relatório	Thiago

Tabela – atuação de cada componente do grupo

1.2 Repositório e organização do código

O código e este relatório estão no repositório do GitHub https://github.com/ThiagoFabiano/bsi-tpa-trabalhos/tree/trabalho2. É o mesmo repositório do trabalho 1 porém o trabalho 2 está todo na branch “trabalho2”. O README do repositório descreve a organização do código e como executá-lo.

1.3 Uso de LLMs no desenvolvimento

O grupo utilizou ferramentas de modelos de linguagem (LLMs), como Gemini e Claude, como apoio durante o desenvolvimento do projeto e a elaboração do relatório. Seu uso esteve voltado principalmente à consulta, revisão, esclarecimento de dúvidas técnicas e apoio à organização do trabalho.

De forma mais detalhada, as ferramentas foram utilizadas para:

- auxiliar na análise e revisão de trechos de código relacionados às etapas 1 e 2, incluindo a ArvoreBinaria, adaptações na Main, ContatoController, ContatoService e os geradores de arquivos;
- auxiliar na identificação e compreensão de erros de compilação relacionados ao uso de tipos genéricos sem parâmetro (raw types), como IColecao em vez de IColecao<Contato>, Medicao sem <T> e List sem <Long>, presentes no ContatoService, na Main e no GeradorArquivosBalanceados;
- apoiar a revisão da lógica da ArvoreBinaria, considerando aspectos como inserção, busca, os três casos de remoção (folha, um filho e dois filhos), cálculo da altura e caminhamentos iterativos. A análise realizada não identificou problemas de lógica nesses pontos;
- apoiar a definição do guia de testes e do protocolo de coleta de tempos utilizados na seção 3;
- auxiliar na elaboração e revisão do relatório, especialmente na organização das análises de complexidade, estruturação das tabelas e revisão textual.
- Criação do arquivo GeradorNomes.java e Makefile

2 Análise matemática de complexidade dos algoritmos

Esta seção analisa os métodos pesquisar, adicionar, remover e quantidadeNos da ArvoreBinaria. Começamos pelo pesquisar, porque o adicionar e o remover fazem a mesma descida pela árvore. As listagens têm as linhas numeradas, e o texto cita esses números. Nas contas, n é o número de contatos e h é a altura da árvore, ou seja, quantas ligações tem o caminho mais longo da raiz até uma folha (uma árvore só com a raiz tem h = 0). Analisamos o pior caso em duas árvores: a degenerada, em que h = n – 1, e a perfeitamente balanceada, em que todos os níveis estão cheios e cada nível tem o dobro de nós do anterior. Somando os níveis:

$$n = 1 + 2 + 4 + ... + 2^h = 2^{(h+1)} - 1, \text{ logo } h = \log_2(n + 1) - 1 \approx \log_2 n$$

Ou seja, na árvore balanceada a altura cresce com o logaritmo de n: dobrar o número de contatos acrescenta só um nível. Nos tamanhos usados na seção 3, o caminho mais longo tem 16 nós com 50.000 contatos e 19 com 400.000.

2.1 Método pesquisar

O pesquisar procura um valor e devolve o objeto guardado na árvore, ou null se não achar. É por devolver o objeto guardado que o ContatoService consegue pesquisar com um Contato que só tem o telefone e receber o contato completo, com o nome.

```
1  @Override
2  public T pesquisar(T valor) {
3     if (valor == null) {
4        return null;
5     }
6     No<T> atual = this.raiz;
7     while (atual != null) {
8        int comp = this.comparador.compare(valor, atual.getValor());
9        if (comp == 0) {
10          return atual.getValor();
11       } else if (comp < 0) {
12          atual = atual.getEsq();
13       } else {
14          atual = atual.getDir();
15       }
16    }
17    return null;
18 }
```

Listagem 1 – método pesquisar

2.1.1 A busca (linhas 3 a 17)

As linhas 3 a 5 recusam o valor nulo e a linha 6 começa pela raiz. A cada volta do while (linhas 7 a 16), o método compara o valor procurado com o do nó atual (linha 8): se forem iguais, devolve o valor (linhas 9 e 10); se for menor, desce para a esquerda (linhas 11 e 12); se for maior, para a direita (linhas 13 e 14). Se chegar a um nó nulo, o valor não está na árvore e a linha 17 devolve null.

Essa é a grande diferença em relação à lista do Trabalho 1: a cada volta, o método escolhe um lado e descarta a outra subárvore inteira. Por isso o número de voltas depende da altura, e não do total de nós: no máximo h + 1. O adicionar e o remover fazem essa mesma descida.

2.1.2 Somando tudo

Linha	O que faz	Quantas vezes roda no pior caso
3 a 6	recusa valor nulo e começa pela raiz	1
7	testa se ainda há nó para olhar	h + 2
8 e 11 a 14	compara com o nó atual e desce um nível	h + 1
9 e 10	devolve o valor se o achou	1 (só quando acha)
17	devolve null se o valor não existe	1 (só quando não acha)

Tabela – contagem de execuções do método pesquisar

$$T(n) = c1 \cdot h + c2, \text{ então o método é } O(h)$$

2.1.3 Pior caso nas duas árvores

Na árvore balanceada, $h \approx \log_2 n$, então o pesquisar é O(log n). O pior caso é buscar uma folha do último nível ou um valor que não existe, o que dá no máximo 16 a 19 comparações nos nossos arquivos. Na árvore degenerada, $h = n - 1$ e o método é O(n): o pior caso é buscar o último contato inserido (o de maior telefone), que obriga a passar pelos n nós. É esse contato que usamos na seção 3. O melhor caso é buscar a raiz, com uma comparação só.

2.1.4 Resumo

Formato da árvore	Altura h	pesquisar no pior caso
Perfeitamente balanceada	$\approx \log_2 n$	O(log n)
Degenerada	n – 1	O(n)
Lista encadeada (Trabalho 1)	—	O(n)

Tabela – ordem de complexidade do pesquisar conforme o formato da árvore

2.2 Método adicionar

O adicionar insere um contato novo no lugar certo. Ele recusa valor nulo e valor repetido, devolvendo false, e nos outros casos desce da raiz até achar uma posição vazia, onde coloca o nó novo como folha. Não há balanceamento: o formato da árvore depende só da ordem em que os contatos chegam.

```
1   @Override
2   public boolean adicionar(T novoValor) {
3      if (novoValor == null) {
4         return false;
5      }
6      No<T> novo = new No<>(novoValor);
7      if (this.raiz == null) {
8         this.raiz = novo;
9         this.quantidade++;
10        return true;
11     }
12     No<T> atual = this.raiz;
13     while (true) {
14        int comp = this.comparador.compare(novoValor, atual.getValor());
15        if (comp == 0) {
16           return false;
17        }
18        if (comp < 0) {
19           if (atual.getEsq() == null) {
20              atual.setEsq(novo);
21              break;
22           }
23           atual = atual.getEsq();
24        } else {
25           if (atual.getDir() == null) {
26              atual.setDir(novo);
27              break;
28           }
29           atual = atual.getDir();
30        }
31     }
32  this.quantidade++;
33  return true;
34  }
```

Listagem 2 – método adicionar

2.2.1 Os casos simples (linhas 3 a 12)

As linhas 3 a 5 recusam o valor nulo e a linha 6 cria o nó novo. Se a árvore está vazia (linha 7), o nó vira a raiz (linhas 8 a 10). A linha 12 começa a descida pela raiz. Tudo isso roda no máximo uma vez.

2.2.2 A descida (linhas 13 a 33)

O while (true) da linha 13 só termina por um return ou um break. Ele faz a mesma descida do pesquisar: compara o valor novo com o do nó atual (linha 14), devolve false se forem iguais (linhas 15 a 17) e vai para a esquerda (linhas 18 a 23) ou para a direita (linhas 24 a 29). Quando o filho daquele lado não existe, o nó novo é colocado ali (linha 20 ou 26) e o laço termina. As linhas 32 e 33 aumentam o contador e devolvem true. Como no pesquisar, o laço dá no máximo h + 1 voltas.

2.2.3 Somando tudo

Linha	O que faz	Quantas vezes roda no pior caso
3 a 6	recusa valor nulo e cria o nó novo	1
7 a 11	trata a árvore vazia	1
12	começa a descida pela raiz	1
13 e 14	repete e compara com o nó atual	h + 1
15 a 17	testa se o valor já existe (linha 15) e, se existir, devolve false (linha 16)	h + 1 (teste); no máximo 1 (return)
18 a 30	escolhe o lado, desce ou pendura o nó novo	h + 1
32 e 33	atualiza o contador e devolve true	1

Tabela – contagem de execuções do método adicionar

Como mostra a tabela, só o laço depende da altura; o resto roda uma vez:

$$T(n) = c1 \cdot h + c2, \text{ então o método é } O(h)$$

2.2.4 Pior caso nas duas árvores

Na árvore balanceada, $h \approx \log_2 n$ e o adicionar é $O(\log n)$. Como todos os lugares vazios ficam abaixo do último nível, qualquer inserção nova desce a árvore inteira. Montar a árvore com n contatos custa $O(n \log n)$, que é o caso dos arquivos balanceados.

Na árvore degenerada, $h = n - 1$ e o adicionar é $O(n)$. Com os telefones em ordem crescente, cada contato novo vai sempre para a direita até o fim. Montar a árvore soma todas essas descidas: $0 + 1 + 2 + \ldots + (n - 1) = n \cdot (n - 1)/2$ comparações, ou seja, $O(n^2)$. Isso dá cerca de 1,25 bilhão de comparações com 50.000 contatos e 80 bilhões com 400.000. O melhor caso é inserir na árvore vazia.

2.2.5 Resumo

Estrutura	adicionar no pior caso	Montagem com n contatos
Árvore perfeitamente balanceada	O(log n)	O(n log n)
Árvore degenerada	O(n)	O(n²)
Lista não ordenada (Trabalho 1)	O(1)	O(n)
Lista ordenada (Trabalho 1)	O(n)	O(n²)

Tabela – ordem de complexidade do adicionar conforme a estrutura

2.3 Método remover

O remover é o método mais longo, porque além de achar o nó precisa tirá-lo sem quebrar a regra da árvore de busca (menores à esquerda, maiores à direita). Ele primeiro busca o nó, guardando também o pai, e depois o retira de um jeito que depende de quantos filhos o nó tem. O trocarFilho (Listagem 4) é um método auxiliar que liga o pai ao nó que fica no lugar do removido.

```
1   @Override
2   public boolean remover(T valor) {
3     if (valor == null) {
4       return false;
5     }
6     No<T> pai = null;
7     No<T> atual = this.raiz;
8     while (atual != null) {
9       int comp = this.comparador.compare(valor, atual.getValor());
10      if (comp == 0) {
11        break;
12      }
13      pai = atual;
14      atual = (comp < 0) ? atual.getEsq() : atual.getDir();
15    }
16    if (atual == null) {
17      return false;
18    }
19
20    if (atual.getEsq() == null && atual.getDir() == null) {
21      trocarFilho(pai, atual, null);
22    } else if (atual.getEsq() == null || atual.getDir() == null) {
23      No<T> filho = (atual.getEsq() != null) ? atual.getEsq() : atual.getDir();
24      trocarFilho(pai, atual, filho);
25    } else {
26        No<T> paiSucessor = atual;
27        No<T> sucessor = atual.getDir();
28        while (sucessor.getEsq() != null) {
29          paiSucessor = sucessor;
30          sucessor = sucessor.getEsq();
31        }
32        atual.setValor(sucessor.getValor());
33        trocarFilho(paiSucessor, sucessor, sucessor.getDir());
34      }
35      this.quantidade--;
36      return true;
37  }
```

Listagem 3 – método remover

```
1 private void trocarFilho(No<T> pai, No<T> antigo, No<T> novo) {
2   if (pai == null) {
3     this.raiz = novo;
4   } else if (pai.getEsq() == antigo) {
5     pai.setEsq(novo);
6   } else {
7     pai.setDir(novo);
8   }
9 }
```

Listagem 4 – método auxiliar trocarFilho

2.3.1 A busca do nó (linhas 3 a 18)

A busca é a mesma do pesquisar, mas guardando o pai do nó atual (linhas 6, 7 e 13), que vai ser usado para religar a árvore. Se o valor não existe, as linhas 16 a 18 devolvem false. O laço dá no máximo h + 1 voltas.

2.3.2 A retirada do nó (linhas 20 a 34)

Quando acha o nó, o método vê quantos filhos ele tem:

- **Caso 1 – folha (linhas 20 e 21):** o trocarFilho coloca null no lugar do nó.
- **Caso 2 – um filho (linhas 22 a 24):** o filho, com toda a sua subárvore, sobe para o lugar do nó.
- **Caso 3 – dois filhos (linhas 25 a 34):** o método copia para o nó o valor do sucessor, que é o menor valor da subárvore direita, e depois retira o sucessor. Para achá-lo, vai uma vez para a direita (linha 27) e depois sempre para a esquerda (linhas 28 a 31). A linha 32 copia o valor e a linha 33 retira o sucessor, que nunca tem filho esquerdo, então essa retirada cai no caso 1 ou 2.

Isso mantém a regra da árvore, porque o sucessor é maior que tudo à esquerda e menor que o resto à direita. Daria para usar o antecessor do mesmo jeito; escolhemos o sucessor.

O trocarFilho não tem laço: se não há pai (linha 2), o nó era a raiz e a raiz muda (linha 3); senão, ele troca o filho do lado certo (linhas 4 a 7). Ele é O(1), assim como as linhas 35 e 36, que diminuem o contador e devolvem true.

2.3.3 Somando tudo

Linha	O que faz	Quantas vezes roda no pior caso
3 a 7	recusa valor nulo e começa a busca pela raiz	1
8	testa se ainda há nó para olhar	h + 2
9 a 14	compara com o nó atual e desce um nível	h + 1
16 a 18	devolve false se o valor não existe	1

Linha	O que faz	Quantas vezes roda no pior caso
20 a 24	casos 1 e 2: religa o pai	1
26, 27, 32 e 33	caso 3: começa a busca do sucessor, copia o valor e religa	1
28 a 31	caso 3: desce até o sucessor	junto com as linhas 8 a 15, no máximo h + 1
35 e 36	atualiza o contador e devolve true	1

Tabela – contagem de execuções do método remover

No caso 3 há dois laços, mas eles não somam 2h voltas: o primeiro desce da raiz até o nó e o

segundo continua dali até o sucessor, formando um único caminho com no máximo h + 1 nós. Então:

$$T(n) = c1 \cdot h + c2, \text{ então o método é } O(h)$$

2.3.4 Pior caso na árvore perfeitamente balanceada

Com $h \approx \log_2 n$:

$$T(n) = c1 \cdot \log_2(n) + c2, \text{ então o método é } O(\log n) \text{ (logarítmico)}$$

O pior caso é remover uma folha do último nível ou procurar um valor que não existe. Remover a raiz custa quase o mesmo, porque o sucessor dela está lá embaixo.

2.3.5 Pior caso na árvore degenerada

Com os telefones em ordem crescente, cada nó tem só o filho direito e a árvore vira, na prática, uma lista:

$$h = n - 1, \text{ então } T(n) = c1 \cdot (n - 1) + c2 \text{ e o método é } O(n) \text{ (linear)}$$

O pior caso é remover o último contato inserido, o de maior telefone, o que obriga a passar pelos n nós. O caso 3 nunca acontece, porque nenhum nó tem dois filhos. O melhor caso é remover a raiz.

2.3.6 Por que o remover usa laço e não recursão

Os métodos da ArvoreBinaria usam laços, e não recursão, porque na árvore degenerada de 400.000 contatos a recursão abriria 400.000 chamadas e o programa pararia com StackOverflowError. A complexidade seria a mesma. A ArvoreAVL da seção 4 usa recursão, mas a altura dela não passa de 25 (seções 4.1 e 4.3).

2.3.7 Resumo

Formato da árvore	Altura h	remover no pior caso
Perfeitamente balanceada	≈ log2 n	O(log n)
Degenerada	n – 1	O(n)
Lista encadeada (Trabalho 1)	—	O(n)

Tabela – ordem de complexidade do remover conforme o formato da árvore

Como mostra a tabela, o código é o mesmo; o que muda é o formato da árvore.

2.4 Método quantidadeNos

O quantidadeNos devolve quantos contatos a árvore guarda (Listagem 5).

```
1 @Override
2 public int quantidadeNos() {
3     return this.quantidade;
4 }
```

Listagem 5 – método quantidadeNos

Ele não percorre a árvore: só devolve o atributo quantidade (linha 3). O adicionar soma 1 quando insere (linhas 9 e 32 da Listagem 2) e o remover subtrai 1 quando retira (linha 35 da Listagem 3). Quando a inserção é recusada ou o valor a remover não existe, os métodos devolvem false antes dessas linhas, então o contador só muda quando a árvore muda.

Como é um único comando, sem laço, nada depende do formato da árvore:

$$T(n) = c, \text{ então o método é } O(1) \text{ (constante)}$$

Isso vale tanto na árvore perfeitamente balanceada quanto na degenerada.

2.5 Resumo dos quatro métodos

Método	Perfeitamente balanceada	Degenerada	Quando ocorre o pior caso
adicionar	O(log n)	O(n)	valor novo que desce até o último nível; na degenerada, valor maior que todos
pesquisar	O(log n)	O(n)	folha do último nível ou valor inexistente; na degenerada, o último inserido
remover	O(log n)	O(n)	folha do último nível ou valor inexistente; na degenerada, o último inserido
quantidadeNos	O(1)	O(1)	não há pior caso: o tempo é sempre o mesmo

Tabela – ordem de complexidade dos métodos no pior caso

3 Análise empírica de complexidade dos algoritmos da biblioteca

Nesta seção medimos o tempo do sistema usando árvores binárias, com arquivos de 50.000, 100.000, 200.000 e 400.000 contatos, os mesmos tamanhos do Trabalho 1. Para cada tamanho há um arquivo com os telefones em ordem crescente (árvore degenerada) e outro organizado para gerar uma árvore perfeitamente balanceada.

3.1 Implementação

3.1.1 Geração dos arquivos de entrada

Cada linha tem o formato nome;telefone. Os geradores GeradorArquivosOrdenados e GeradorArquivosBalanceados (pacote util) foram adaptados dos exemplos do professor: em vez de alunos com matrícula, geram contatos. O telefone é 27900000000 mais um índice de 0 a n – 1, e o nome é sorteado pela classe GeradorNomes, sem repetição.

Arquivo ordenado. Os índices são escritos em ordem crescente, então cada telefone vai para a direita do anterior e a árvore fica degenerada.

Arquivo balanceado. O método recursivo dividirEConquistar escreve primeiro o índice do meio do intervalo, que vira a raiz daquele trecho, e depois repete o processo nas duas metades. Inserida nessa ordem, a árvore fica perfeitamente balanceada, com as alturas da tabela abaixo.

Tamanho	Altura da árvore balanceada	Nós no caminho mais longo	Altura da árvore degenerada
50.000	15	16	49.999
100.000	16	17	99.999
200.000	17	18	199.999
400.000	18	19	399.999

Tabela – alturas esperadas das árvores de telefone

Conferimos: com os arquivos balanceados, o método altura() da ArvoreBinaria deu 15, 16, 17 e 18, e nos oito arquivos as duas árvores ficaram com o mesmo número de nós do arquivo.

Nomes únicos. A árvore recusa chaves repetidas: com os 440 nomes da versão antiga do gerador, a árvore de nomes ficava com só 440 nós. Com nomes únicos ela guarda os n contatos, e como os nomes chegam em ordem aleatória, sua altura ficou entre 34 e 47.

3.1.2 Definição do elemento de pior caso

O enunciado pede a busca pela folha mais distante da raiz na árvore de telefones. Usamos o mesmo contato nas três medidas (pesquisa por telefone, pesquisa pelo nome dele e remoção):

- **Arquivo ordenado:** o contato de maior telefone, o último do arquivo, que fica no fim da árvore degenerada.
- **Arquivo balanceado:** uma folha do último nível. O gerador mostra no terminal o primeiro telefone que escreve na maior profundidade: 27900000001 em todos os tamanhos.

Arquivo	Telefone de pior caso	Nome do contato
ordenado, 50.000	27900049999	Igor Paz Queiroz
ordenado, 100.000	27900099999	Renato Abreu Vasconcelos
ordenado, 200.000	27900199999	Jorge Xisto Lima
ordenado, 400.000	27900399999	Emanuel Iglesias Lima
balanceado, 50.000	27900000001	Mauricio Porto Werneck
balanceado, 100.000	27900000001	Osvaldo Vilela Delgado
balanceado, 200.000	27900000001	Gabriel Dantas Xisto
balanceado, 400.000	27900000001	Renata Noronha Rangel

Tabela – elemento de pior caso de cada arquivo

3.1.3 O que cada tempo mede

A montagem é o tempo do carregarArquivo inteiro: ler o arquivo e inserir cada contato nas duas árvores (nome e telefone). Nas outras medidas, só a chamada ao método da árvore é cronometrada, com System.nanoTime(). Na remoção, a pesquisa que vem antes e a retirada da árvore de nomes ficam fora da medição.

3.1.4 Procedimento de coleta

Item	Valor
Processador	AMD Ryzen 7 5825U (8 núcleos, 16 threads, até 4,5 GHz)
Memória RAM	16 GB
Sistema operacional	Ubuntu 24.04.4 LTS
Versão do Java	OpenJDK 21.0.12.1
Mesma máquina do Trabalho 1	Sim

Tabela – ambiente de teste

Para cada um dos oito arquivos, iniciamos o programa numa JVM nova com a opção 3 (árvore binária), carregamos o arquivo (opção 1), pesquisamos duas vezes seguidas o telefone de pior caso (opção 4) e o nome dele (opção 3), removemos o telefone (opção 5) e saímos (opção 7).

Cada arquivo foi executado 3 vezes, e as tabelas trazem a mediana; um script digitou as opções do menu. A primeira pesquisa inclui o aquecimento da JVM (213.352 ns contra 3.086 ns na segunda, na balanceada de 400.000), por isso as tabelas usam a segunda. A remoção não dá para repetir e inclui esse custo.

3.2 Análise de complexidade

3.2.1 Tempos medidos

Tamanho	Altura h	Montagem (ms)	Pesquisa por telefone (ns)	Pesquisa por nome (ns)	Remoção por telefone (ns)
50.000	15	95	2.776	25.728	7.875
100.000	16	161	3.296	27.362	7.825
200.000	17	307	3.086	26.108	8.326
400.000	18	662	3.086	15.850	8.526

Tabela – tempos medidos na árvore balanceada (mediana de 3 execuções)

Tamanho	Altura h	Montagem (s)	Pesquisa por telefone (ns)	Pesquisa por nome (ns)	Remoção por telefone (ns)
50.000	49.999	14,6	1.756.150	38.563	3.244.465

Tamanho	Altura h	Montagem (s)	Pesquisa por telefone (ns)	Pesquisa por nome (ns)	Remoção por telefone (ns)
100.000	99.999	99,2	4.553.259	27.181	9.252.237
200.000	199.999	560,7	9.958.482	28.243	15.589.482
400.000	399.999	3.060,5	18.985.797	29.326	24.554.280

Tabela – tempos medidos na árvore degenerada (mediana de 3 execuções)

3.2.2 O que esperamos da seção 2

Como o tamanho dobra de um arquivo para o outro, a razão entre dois tempos seguidos indica a complexidade: perto de 1, tempo quase constante; perto de 2, linear; perto de 4, quadrático. A tabela mostra o que a seção 2 prevê.

Medida	Balanceada	Razão esperada	Degenerada	Razão esperada
Montagem	O(n log n)	≈ 2,1	O(n²)	≈ 4
Pesquisa por telefone	O(log n)	≈ 1,06	O(n)	≈ 2
Remoção por telefone	O(log n)	≈ 1,06	O(n)	≈ 2
Pesquisa por nome	ver 3.2.4	—	ver 3.2.4	—

Tabela – comportamento esperado ao dobrar o tamanho do arquivo

O 1,06 é só uma comparação a mais (de 16 para 17), então esperamos que esses tempos apenas oscilem.

3.2.3 Razões medidas

Medida	Árvore	50 mil → 100 mil	100 mil → 200 mil	200 mil → 400 mil
Montagem	balanceada	1,69	1,91	2,16
Montagem	degenerada	6,79	5,65	5,46
Pesquisa por telefone	balanceada	1,19	0,94	1,00
Pesquisa por telefone	degenerada	2,59	2,19	1,91
Remoção por telefone	balanceada	0,99	1,06	1,02
Remoção por telefone	degenerada	2,85	1,68	1,58
Pesquisa por nome	balanceada	1,06	0,95	0,61
Pesquisa por nome	degenerada	0,70	1,04	1,04

Tabela – razão entre os tempos de tamanhos consecutivos

Árvore balanceada. Como mostra a tabela, a montagem só chegou ao ≈ 2,1 de O(n log n) na última dobra (as duas primeiras ficaram abaixo), e a pesquisa e a remoção ficaram perto de 1, como previsto para O(log n). Esses tempos são muito pequenos (3 a 8 µs); achamos que quase tudo é custo fixo da chamada, com o código ainda pouco otimizado pela JVM, e não as comparações. O que mostra o O(log n) é o tempo não crescer com n.

Árvore degenerada. A pesquisa por telefone ficou perto do ≈ 2 de O(n). A remoção cresceu 7,6 vezes para um arquivo 8 vezes maior, também linear; as razões dela variam mais, provavelmente porque é a primeira chamada do remover e inclui o aquecimento da JVM.

Na montagem degenerada, as razões passaram de 4: as comparações seguem $n \cdot (n - 1)/2$, mas cada uma ficou mais cara (cerca de 12 ns com 50.000 contatos e 38 ns com 400.000). Nossa hipótese é a memória cache, que comporta boa parte dos dados com 50.000 contatos, mas não com 400.000; não medimos isso. Pela contagem de comparações o algoritmo continua O(n²), mas o tempo medido cresceu mais que isso: cerca de 210 vezes para um arquivo 8 vezes maior, contra 64 do O(n²).

3.2.4 A pesquisa por nome

A pesquisa por nome roda na árvore de nomes, que não degenera, porque os nomes chegam em ordem aleatória, e o contato escolhido não é o pior caso dela. Como mostram as tabelas, ela ficou entre 15,8 e 38,6 µs em todos os arquivos, sem crescer com n. Na balanceada, foi de 5 a 9 vezes mais lenta que a pesquisa por telefone, mesmo passando por menos nós (o contato fica na profundidade 4 a 8 da árvore de nomes, contra 15 a 18 na de telefones). Achamos que pesa mais o custo da chamada, como o compareToIgnoreCase e o aquecimento da JVM, mas não medimos isso.

3.2.5 Comparação com as listas do Trabalho 1

No Trabalho 1 medimos os mesmos tempos com listas, nos mesmos tamanhos, com a busca e a remoção também no pior caso. As tabelas juntam esses tempos, copiados do relatório do Trabalho 1, com os das árvores. Como no Trabalho 1 cada pesquisa foi medida uma vez só, aqui usamos a primeira pesquisa das árvores.

Tamanho	Lista não ordenada (ms)	Lista ordenada (s)	Árvore balanceada (ms)	Árvore degenerada (s)
50.000	43,4	46,93	95	14,6
100.000	91,1	205,99	161	99,2

Tamanho	Lista não ordenada (ms)	Lista ordenada (s)	Árvore balanceada (ms)	Árvore degenerada (s)
200.000	128,2	886,21	307	560,7
400.000	171,8	4.176,89	662	3.060,5

Tabela – tempo de montagem: listas e árvores

Tamanho	Lista não ordenada (ns)	Lista ordenada (ns)	Árvore balanceada (ns)	Árvore degenerada (ns)
50.000	5.432.309	9.141.336	219.564	3.416.913
100.000	7.446.970	10.828.974	227.037	10.566.543
200.000	8.097.667	17.345.698	264.276	15.970.010
400.000	10.779.846	34.486.922	213.352	24.577.554

Tabela – pesquisa de pior caso pelo telefone: listas e árvores

Tamanho	Lista não ordenada (ns)	Lista ordenada (ns)	Árvore balanceada (ns)	Árvore degenerada (ns)
50.000	4.269.648	9.174.849	7.875	3.244.465
100.000	6.034.364	9.702.087	7.825	9.252.237
200.000	7.338.313	16.827.141	8.326	15.589.482
400.000	10.043.767	35.195.429	8.526	24.554.280

Tabela – remoção de pior caso pelo telefone: listas e árvores

Montagem. A árvore degenerada e a lista ordenada fazem cerca de $n^2/2$ comparações, $O(n^2)$, e levam segundos; a degenerada foi de 1,4 a 3,2 vezes mais rápida, mas na mesma ordem de grandeza. A árvore balanceada é O(n log n) e leva milissegundos, mas foi de 1,8 a 3,9 vezes

mais lenta que a lista não ordenada, que só coloca o contato no fim, sem comparar.

Pesquisa e remoção. Nas listas e na árvore degenerada as duas operações são O(n); na balanceada, O(log n), com no máximo 19 comparações. Com 400.000 contatos, a remoção na balanceada foi cerca de 1.178 vezes mais rápida que na lista não ordenada, e a degenerada ficou na mesma ordem das listas. Nas listas, a pesquisa cresceu menos que o arquivo (de 5,4 para 10,8 ms na não ordenada), provavelmente pelo aquecimento da JVM, já que no Trabalho 1 cada pesquisa foi medida uma vez só.

Conclusão da comparação: a árvore só é melhor que a lista quando está balanceada. Com o arquivo ordenado, ela se comportou como uma lista. Como não dá para controlar a ordem dos dados, precisamos de uma árvore que se balanceie sozinha, que é a árvore AVL da próxima parte.

4 Árvore AVL: análise matemática de complexidade

A ArvoreAVL herda da ArvoreBinaria e sobrescreve só o adicionar e o remover; o pesquisar, o quantidadeNos, a altura e os caminhamentos são herdados. A diferença é que, depois de inserir ou remover, a AVL volta pelo caminho percorrido e balanceia cada nó. As convenções são as da seção 2 (e a árvore vazia tem h = –1).

4.1 Por que a altura da AVL é O(log n)

Na AVL, em todo nó o fator de balanceamento FB = altura(esq) – altura(dir) fica entre –1 e +1. Isso impede a árvore degenerada, mas falta mostrar que a altura é mesmo logarítmica no pior formato permitido. Para isso, pensamos na menor AVL possível com altura h: um filho tem altura h – 1 e o outro h – 2, então o número mínimo de nós é N(h) = N(h – 1) + N(h – 2) + 1, uma conta parecida com a de Fibonacci.

h	**0**	**1**	**2**	**3**	**4**	**5**	**6**
menor nº de nós N(h)	1	2	4	7	12	20	33

Tabela – menor quantidade de nós de uma AVL de altura h

Como mostra a tabela, N(h) mais que dobra a cada dois níveis. Continuando a conta, N(25) = 317.810 e N(26) = 514.228: uma AVL de altura 26 precisa de pelo menos 514.228 nós, então com 400.000 nós a altura é no máximo 25. Em geral, uma AVL com n nós tem altura menor que 1,44·log2(n + 2). E nenhuma árvore com n nós é mais baixa que a perfeitamente balanceada. Então a altura da AVL sempre fica entre cerca de log2 n e 1,44·log2 n, ou seja, cresce como log n, não importa a ordem dos dados.

Tamanho n	**h mínimo (perfeita)**	**h máximo (AVL mais torta)**
50.000	15	21
100.000	16	22
200.000	17	24
400.000	18	25

Tabela – faixa de altura da AVL nos tamanhos de arquivo usados

Na prática, com os telefones em ordem crescente, a ArvoreAVL ficou com altura 15, 16, 17 e 18, o menor valor possível (conferido também na seção 5).

4.2 Métodos de balanceamento

Os métodos de balanceamento usam a altura guardada em cada nó (campo altura do No), então nenhum deles precisa percorrer a árvore.

```
 1  private void atualizarAlturaNo(No<T> no) {
 2      int novaAltura = 1 + Math.max(alturaNo(no.getEsq()), alturaNo(no.getDir()));
 3      no.setAltura(novaAltura);
 4  }
 5  private int alturaNo(No<T> no) {
 6      if (no == null) {
 7          return -1;
 8      }
 9      return no.getAltura();
10  }
11   private int fatorBalanceamento(No<T> no) {
12       int fb = alturaNo(no.getEsq()) - alturaNo(no.getDir());
13       return fb;
14  }
```

Listagem 6 – atualizarAlturaNo, alturaNo e fatorBalanceamento

O alturaNo (linhas 5 a 10) devolve –1 para nó nulo e, senão, a altura guardada; assim, uma folha tem altura 1 + max(–1, –1) = 0. O atualizarAlturaNo (linhas 1 a 4) recalcula a altura do nó a partir dos filhos, e o fatorBalanceamento (linhas 11 a 14) faz a diferença entre as alturas. Nenhum tem laço, então os três são O(1).

```
1  private No<T> rotacaoDireita(No<T> noDesbalanceado) {
2      No<T> novoNoPai = noDesbalanceado.getEsq();
3      noDesbalanceado.setEsq(novoNoPai.getDir());
4      novoNoPai.setDir(noDesbalanceado);
5      atualizarAlturaNo(noDesbalanceado);
6      atualizarAlturaNo(novoNoPai);
7      return novoNoPai;
8  }
```

Listagem 7 – rotacaoDireita (a rotacaoEsquerda é o espelho)

Na rotacaoDireita, o filho esquerdo sobe (linha 2), a subárvore direita dele passa a ser a esquerda do nó que desce (linha 3), e o nó que desce vira filho direito do que subiu (linha 4). As linhas 5 e 6 atualizam as alturas, primeiro a do nó de baixo, e a linha 7 devolve o novo topo. Sem laço, a rotação é O(1).

```
 1  private No<T> balancear(No<T> no) {
 2      atualizarAlturaNo(no);
 3      int fb = fatorBalanceamento(no);
 4      if (fb > 1) {
 5          if (fatorBalanceamento(no.getEsq()) < 0) {
 6              no.setEsq(rotacaoEsquerda(no.getEsq()));
 7          }
 8          return rotacaoDireita(no);
 9      }
10       if (fb < -1) {
11           if (fatorBalanceamento(no.getDir()) > 0) {
12               no.setDir(rotacaoDireita(no.getDir()));
13           }
14           return rotacaoEsquerda(no);
```

```
15      }
16      return no;
17  }
```

Listagem 8 – balancear

O balancear atualiza a altura e calcula o FB (linhas 2 e 3). Se FB > 1, a árvore pesa para a esquerda (linhas 4 a 9): se o filho esquerdo pesa para a direita (linha 5), primeiro ele gira para a esquerda (linha 6), e depois o nó gira para a direita (linha 8). As linhas 10 a 15 fazem o mesmo, espelhado, para FB < –1. Sem desbalanceamento, a linha 16 devolve o próprio nó. São no máximo duas rotações, então o balancear é O(1). Como a rotação muda o topo da subárvore, quem chama precisa usar o valor devolvido, como fazem as linhas 6, 8, 12 e 14.

Caso	FB do nó	FB do filho pesado	Correção
Esquerda-esquerda	+2	+1 ou 0	rotação à direita no nó
Direita-direita	–2	–1 ou 0	rotação à esquerda no nó
Esquerda-direita	+2	–1	esquerda no filho, depois direita no nó
Direita-esquerda	–2	+1	direita no filho, depois esquerda no nó

Tabela – os quatro casos de balanceamento

O FB 0 no filho só aparece na remoção; por isso as linhas 5 e 11 usam < 0 e > 0, e nesse caso basta a rotação simples.

4.3 Método adicionar

O adicionar da AVL tem um método público (Listagem 9), que trata os casos simples e começa a recursão, e um recursivo (Listagem 10), que faz o trabalho.

```
1  @Override
2  public boolean adicionar(T novoValor) {
3      if (novoValor == null) {
4          return false;
5      }
6      int quantidadeAntes = this.quantidade;
7      this.raiz = adicionar(this.raiz, novoValor);
8      return this.quantidade > quantidadeAntes;
9  }
```

Listagem 9 – adicionar (público)

```
1  private No<T> adicionar(No<T> no, T valor) {
2      if (no == null) {
3          No<T> novoNo = new No<>(valor);
4          this.quantidade++;
5          return novoNo;
6      }
7      int comp = this.comparador.compare(valor, no.getValor());
8      if (comp == 0) {
9          return no;
```

```
10      }
11      if (comp < 0) {
12          no.setEsq(adicionar(no.getEsq(), valor));
13      } else {
14          no.setDir(adicionar(no.getDir(), valor));
15      }
16      return balancear(no);
17  }
```

Listagem 10 – adicionar recursivo

No público, as linhas 3 a 5 recusam o valor nulo, a linha 6 guarda a quantidade, a linha 7 chama o recursivo e guarda o resultado como nova raiz (uma rotação pode trocar a raiz) e a linha 8 devolve true se a quantidade aumentou. Tudo O(1).

No recursivo, o nó nulo é o caso base (linhas 2 a 6): achou o lugar vazio, cria o nó e soma 1 na quantidade. Se o valor já existe (linhas 8 a 10), devolve o próprio nó, sem inserir; devolver null apagaria a subárvore, porque o pai religa o retorno. Senão, desce para a esquerda ou para a direita (linhas 11 a 15) e, na volta, a linha 16 balanceia cada nó do caminho.

Linhas (Listagem 10)	O que faz	Quantas vezes no pior caso
2 a 6	cria o nó novo	1
7	compara com o nó atual	h + 1
8 a 10	trata o valor duplicado	1 (só se existir)
11 a 15	desce um nível e religa	h + 1
16	balancear (O(1))	h + 1

Tabela – contagem de execuções do adicionar recursivo

Como mostra a tabela, cada chamada faz uma quantidade fixa de comandos, e são no máximo h + 2 chamadas. Logo:

T(n) = c1·h + c2, então o método é O(h). Como a altura da AVL é O(log n) (seção 4.1), o adicionar é O(log n).

Na inserção acontece no máximo uma rotação (simples ou dupla), que devolve a subárvore à altura de antes; os nós acima só atualizam a altura. O pior caso é inserir um valor que vai para o último nível e ainda causa uma rotação, mas continua O(log n). A árvore degenerada não existe na AVL: tanto no pior formato permitido quanto no melhor, a altura é O(log n).

Montar a árvore com n inserções custa O(n log n), mesmo com o arquivo ordenado, contra O(n²) na árvore comum. A recursão empilha no máximo h + 2 chamadas, 27 com 400.000 contatos (20 com o arquivo ordenado), então, ao contrário da árvore comum degenerada, não há risco de StackOverflowError. O espaço extra é um int por nó, mais a pilha da recursão.

4.4 Método remover

A remoção também tem uma versão pública (Listagem 11) e uma recursiva (Listagem 12).

```
1  @Override
2  public boolean remover(T valor) {
3      if (valor == null) {
4          return false;
```

```
5     }
6     int quantidadeAntes = this.quantidade;
7     this.raiz = remover(this.raiz, valor);
8     return this.quantidade < quantidadeAntes;
9  }
```

Listagem 11 – remover (público)

```
1  private No<T> remover(No<T> no, T valor) {
2     if (no == null) {
3        return null;
4     }
5     int comp = this.comparador.compare(valor, no.getValor());
6     if (comp < 0) {
7        no.setEsq(remover(no.getEsq(), valor));
8     } else if (comp > 0) {
9        no.setDir(remover(no.getDir(), valor));
10    } else {
11       if (no.getEsq() == null) {
12          this.quantidade--;
13          return no.getDir();
14       }
15       if (no.getDir() == null) {
16          this.quantidade--;
17          return no.getEsq();
18       }
19       No<T> sucessor = no.getDir();
20       while (sucessor.getEsq() != null) {
21          sucessor = sucessor.getEsq();
22       }
23       no.setValor(sucessor.getValor());
24       no.setDir(remover(no.getDir(), sucessor.getValor()));
25    }
26    return balancear(no);
27 }
```

Listagem 12 – remover recursivo

O público (Listagem 11) é igual ao do adicionar, mas devolve true se a quantidade diminuiu. No recursivo (Listagem 12), chegar a um nó nulo (linhas 2 a 4) quer dizer que o valor não existe, e nada muda. A descida (linhas 6 a 9) é igual à do adicionar.

Quando acha o valor, os casos são os mesmos da seção 2.3: sem filho esquerdo ou sem filho direito (linhas 11 a 18), devolve o outro filho e diminui a quantidade; com dois filhos (linhas 19 a 24), acha o sucessor (linhas 19 a 22), copia o valor dele (linha 23) e o remove da subárvore direita (linha 24). Na volta, a linha 26 balanceia os nós do caminho.

Linhas (Listagem 12)	O que faz	Quantas vezes no pior caso
2 a 4	valor inexistente	1
5 a 9	compara e desce um nível	h + 1
10 a 18	remove nó sem filho ou com um filho	1

Linhas (Listagem 12)	O que faz	Quantas vezes no pior caso
19 a 24	dois filhos: sucessor e remoção dele	1 trecho; o while (20 a 22) desce até o sucessor e a chamada da linha 24 percorre esse trecho de novo
26	balancear (O(1))	h + 1

Tabela – contagem de execuções do remover recursivo

A busca desce no máximo h + 1 níveis. Com dois filhos, o trecho entre o nó e o sucessor é percorrido duas vezes: pelo while (linhas 20 a 22) e pela chamada da linha 24. São no máximo uns 2h + 1 nós, o que continua proporcional a h:

T(n) = c1·h + c2, então o método é O(h) = O(log n).

A diferença para a inserção é que a remoção pode precisar de rotação em vários níveis, porque uma rotação pode diminuir a altura da subárvore e passar o desequilíbrio para cima. Cada rotação é O(1) e há no máximo um rebalanceamento (simples ou duplo) por nível, então o total continua O(log n). O pior caso é remover um nó fundo numa AVL de formato mínimo (a árvore de Fibonacci da seção 4.1), em que as rotações se repetem a cada dois níveis até a raiz: cerca de h/2 rotações, além da descida. Na árvore comum degenerada o remover era O(n); na AVL, o pior caso não depende mais da ordem dos dados.

4.5 quantidadeNos e pesquisar (herdados)

O quantidadeNos continua O(1) (seção 2.4): o contador é mantido pelo adicionar (Listagem 10, linha 4) e pelo remover (Listagem 12, linhas 12 e 16). O pesquisar usa o mesmo laço da seção 2.1, O(h), que na AVL vira O(log n) em qualquer ordem de entrada.

4.6 Resumo

Método	AVL (pior caso)	Árvore comum balanceada	Árvore comum degenerada	Pior caso na AVL
adicionar	O(log n)	O(log n)	O(n)	valor novo no último nível, com 1 rotação
remover	O(log n)	O(log n)	O(n)	nó profundo, com rotações em cascata até a raiz
pesquisar	O(log n)	O(log n)	O(n)	folha do último nível ou valor inexistente
quantidadeNos	O(1)	O(1)	O(1)	não há pior caso
montagem com n contatos	O(n log n)	O(n log n)	O(n²)	—

Tabela – ordem de complexidade no pior caso: AVL contra árvore comum

Na AVL, o formato da árvore não depende mais da ordem dos dados, e por isso todos os métodos ficam O(log n), mesmo com o arquivo ordenado.

5 Análise empírica da árvore AVL

5.1 Implementação

O programa ganhou a opção 4 (Árvore AVL) na ContatoView e na Main; o resto não mudou, porque o service e o controller só usam a interface IColecao. Usamos os arquivos ordenados da seção 3 e o mesmo contato de pior caso (o último do arquivo), que na AVL de telefones é uma folha do último nível. A coleta seguiu a seção 3.1.4, mas com a opção 4 e 10 execuções por arquivo em vez de 3, porque a pesquisa e a remoção levam poucos microssegundos e a mediana de 10 oscila menos.

Item	Valor
Processador	AMD Ryzen 7 5825U (8 núcleos, 16 threads, até 4,5 GHz)
Memória RAM	16 GB
Sistema operacional	Ubuntu 24.04.5 LTS (na seção 3: 24.04.4)
Versão do Java	OpenJDK 21.0.12.1 (Temurin)
Mesma máquina da seção 3	Sim

Tabela – ambiente de teste

5.2 Análise de complexidade

5.2.1 Tempos medidos

Tamanho	Altura h	Montagem (ms)	Pesquisa por telefone (ns)	Pesquisa por nome (ns)	Remoção por telefone (ns)
50.000	15	105	3.832	27.708	11.262
100.000	16	185	3.272	27.271	10.570
200.000	17	339	2.985	22.212	10.360
400.000	18	735	3.036	21.831	11.026

Tabela – tempos medidos na AVL com arquivos ordenados (mediana de 10 execuções)

Nas 40 rodadas, as duas árvores ficaram com o número de contatos do arquivo. A primeira pesquisa por telefone levou de 228 a 318 µs, contra cerca de 3 µs na segunda, por causa do aquecimento da JVM, e a segunda variou entre 2,4 e 5,9 µs de uma execução para outra.

5.2.2 O que esperamos da seção 4

Medida	AVL	Razão esperada ao dobrar n
Montagem	O(n log n)	≈ 2,1
Pesquisa por telefone	O(log n)	≈ 1,06 (tempo quase constante)
Remoção por telefone	O(log n)	≈ 1,06 (tempo quase constante)
Pesquisa por nome	O(log n)	≈ 1,06

Tabela – comportamento esperado da AVL ao dobrar o tamanho do arquivo

As previsões são as mesmas da árvore balanceada (seção 3.2.2). Na pesquisa por nome, a árvore de nomes da AVL deveria ficar bem mais baixa que a da árvore comum (34 a 47), e ficou: alturas 18, 19, 20 e 22. O contato de pior caso fica na profundidade 13, 17, 18 e 19 na árvore de nomes da AVL, e em 21, 25, 14 e 22 na árvore de nomes comum.

5.2.3 Razões medidas

Medida	50 mil → 100 mil	100 mil → 200 mil	200 mil → 400 mil
Montagem	1,76	1,83	2,17
Pesquisa por telefone	0,85	0,91	1,02
Remoção por telefone	0,94	0,98	1,06

Medida	50 mil → 100 mil	100 mil → 200 mil	200 mil → 400 mil
Pesquisa por nome	0,98	0,81	0,98

Tabela – razão entre os tempos de tamanhos consecutivos (AVL)

Como mostra a tabela, na montagem só a última razão (2,17) bate com o ≈ 2,1 de O(n log n); as outras ficaram abaixo, como na árvore balanceada. Achamos que uma parte fixa, como abrir o arquivo e o aquecimento da JVM, pesa mais nos arquivos pequenos, mas não medimos isso.

A pesquisa por telefone e a remoção oscilam em torno de 1, sem tendência: a comparação a mais por dobra é menor que a variação entre execuções, e o que os dados mostram é que o tempo não cresce com n. Na pesquisa por nome, o tempo até caiu (de 27,7 para 21,8 µs) enquanto a profundidade do contato subiu de 13 para 19; como na seção 3.2.4, achamos que pesa mais o custo da chamada do que as comparações.

5.2.4 Comparação com a árvore binária comum (mesmos arquivos)

As tabelas colocam lado a lado os dados das seções 3.2.1 e 5.2.1. A degenerada usou o mesmo arquivo e o mesmo contato da AVL; a balanceada usou outro arquivo, então a comparação com ela é aproximada.

Tamanho	Árvore degenerada (s)	Árvore balanceada (ms)	AVL (ms)
50.000	14,6	95	105
100.000	99,2	161	185
200.000	560,7	307	339
400.000	3.060,5	662	735

Tabela – montagem: árvore comum e AVL (arquivos ordenados)

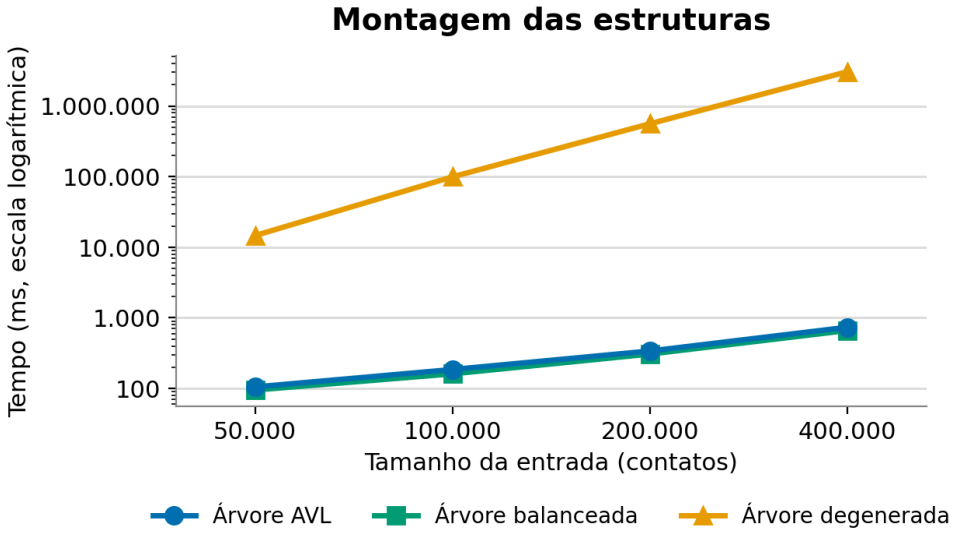


Figura 1 – Montagem: AVL, árvore balanceada e árvore degenerada (ms, eixo y em escala logarítmica; cada tamanho no eixo x é o dobro do anterior)

Tamanho	Árvore degenerada (ns)	Árvore balanceada (ns)	AVL (ns)
50.000	1.756.150	2.776	3.832
100.000	4.553.259	3.296	3.272
200.000	9.958.482	3.086	2.985

Tamanho	Árvore degenerada (ns)	Árvore balanceada (ns)	AVL (ns)
400.000	18.985.797	3.086	3.036

Tabela – pesquisa de pior caso pelo telefone: árvore comum e AVL

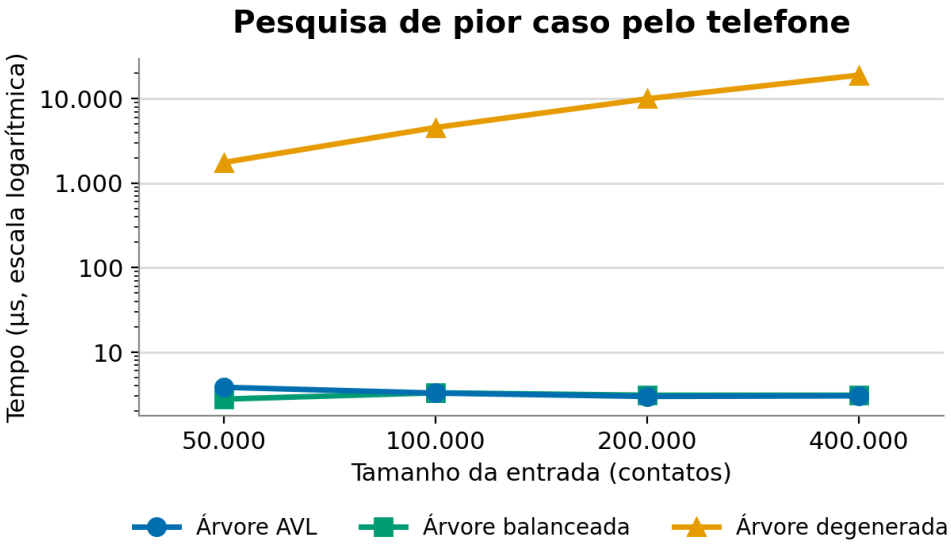


Figura 2 – Pesquisa de pior caso pelo telefone (µs, eixo y em escala logarítmica; cada tamanho no eixo x é o dobro do anterior)

Tamanho	Árvore degenerada (ns)	Árvore balanceada (ns)	AVL (ns)
50.000	3.244.465	7.875	11.262
100.000	9.252.237	7.825	10.570
200.000	15.589.482	8.326	10.360
400.000	24.554.280	8.526	11.026

Tabela – remoção de pior caso pelo telefone: árvore comum e AVL

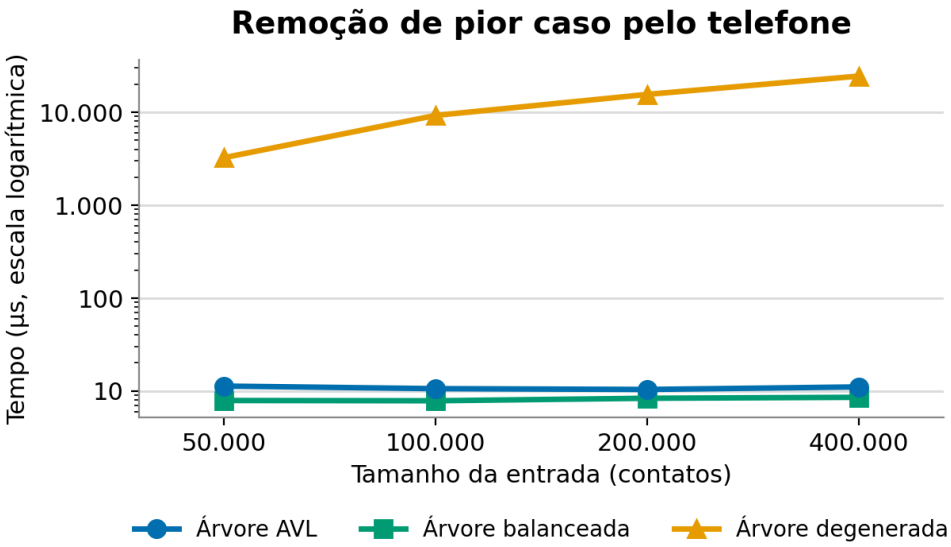


Figura 3 – Remoção de pior caso pelo telefone (µs, eixo y em escala logarítmica; cada tamanho no eixo x é o dobro do anterior)

Tamanho	Árvore degenerada (ns)	Árvore balanceada (ns)	AVL (ns)
50.000	38.563	25.728	27.708
100.000	27.181	27.362	27.271
200.000	28.243	26.108	22.212
400.000	29.326	15.850	21.831

Tabela – pesquisa por nome: árvore comum e AVL

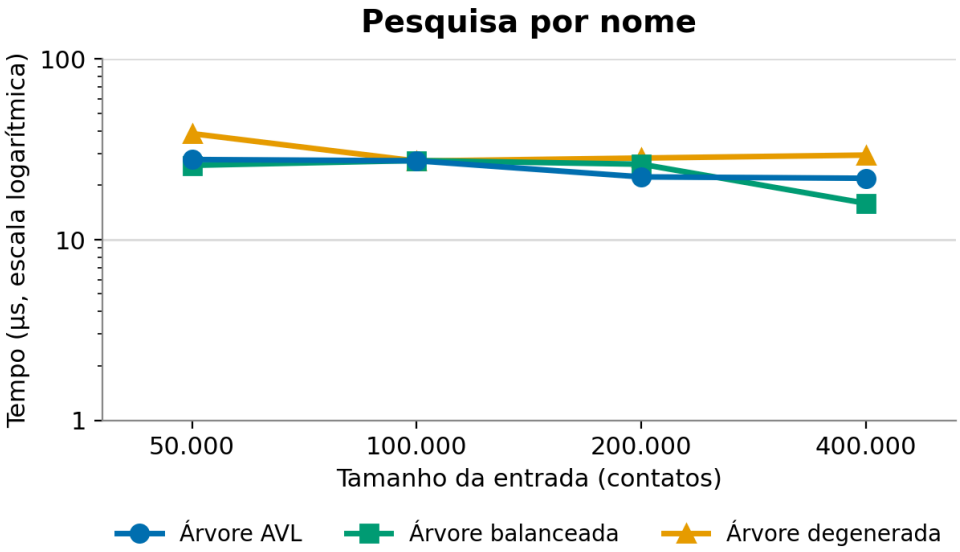


Figura 4 – Pesquisa por nome (µs, eixo y em escala logarítmica; cada tamanho no eixo x é o dobro do anterior)

As figuras mostram o resultado principal: nas três operações com o telefone, a AVL ficou junto da árvore balanceada e muito abaixo da degenerada. Na montagem, a AVL levou 735 ms com 400.000 contatos, contra 3.060,5 s da degenerada, e foi de 10% a 15% mais lenta que a balanceada, porque atualiza alturas e faz rotações. Na pesquisa por telefone, com a mesma altura

da balanceada, os tempos ficaram praticamente iguais (só com 50.000 a AVL foi 38% mais lenta, dentro da variação entre execuções); na remoção, a AVL levou de 1,24 a 1,43 vez o tempo da balanceada, porque ainda balanceia na volta da recursão. Na pesquisa por nome, as três árvores ficaram na mesma faixa (Figura 4).

O ponto principal é que a entrada que estragava a árvore comum, os telefones em ordem crescente, deixa de ser um problema. Contra a degenerada, a AVL foi de 139 a 4.164 vezes mais rápida na montagem, de 458 a 6.254 vezes na pesquisa por telefone e de 288 a 2.227 vezes na remoção, e a vantagem cresce com n. Contra a balanceada, ficou um pouco mais lenta na montagem e na remoção e praticamente igual na pesquisa, salvo com 50.000 contatos.

6 Árvores binárias na biblioteca padrão do Java

O pacote java.util tem duas classes baseadas em árvore binária de busca: TreeMap e TreeSet.

6.1 Estrutura de dados

O TreeMap<K,V> guarda pares chave–valor ordenados pela chave. O Javadoc o define como "A Red-Black tree based NavigableMap implementation", ou seja, ele é uma árvore rubro-negra. O TreeSet<E> não tem árvore própria: por dentro ele usa um TreeMap, em que cada elemento é uma chave e o valor é sempre o mesmo objeto fixo (chamado PRESENT no código-fonte).

Cada nó (a classe interna TreeMap.Entry) guarda a chave, o valor, os filhos esquerdo e direito, o pai e a cor (campo boolean color). Diferente do nosso No, ele tem ponteiro para o pai, porque o TreeMap sobe pela árvore com laços em vez de recursão.

6.2 Balanceamento

A árvore rubro-negra segue quatro regras: todo nó é vermelho ou preto; a raiz é preta; um nó vermelho não tem filho vermelho; todos os caminhos de um nó até um filho nulo passam pelo mesmo número de nós pretos. Com isso, o caminho mais longo tem no máximo o dobro do mais curto, e a altura fica $h \leq 2 \cdot \log_2(n + 1)$. É um balanceamento mais folgado que o da AVL (no máximo cerca de $1{,}44 \cdot \log_2 n$, seção 4.1), mas a altura continua logarítmica.

Depois de uma inserção, o fixAfterInsertion sobe pelo caminho recolorindo nós e faz no máximo duas rotações; depois de uma remoção, o fixAfterDeletion faz no máximo três. As rotações são as mesmas rotações simples da nossa AVL.

6.3 Comparação dos elementos

No TreeMap há duas formas de comparar: passar um Comparator no construtor ou, sem Comparator, usar a ordem natural do tipo (a interface Comparable); a nossa árvore usa só o Comparator. Se não há Comparator e a chave não implementa Comparable, o put lança ClassCastException, o que acontece já na primeira inserção.

Duas chaves são consideradas iguais quando o compare (ou o compareTo) devolve 0, e não pelo equals. Se a chave já existe, o TreeMap.put mantém a chave antiga, troca o valor e devolve o valor anterior. Já o TreeSet.add não muda nada e devolve false, como o adicionar da nossa árvore (seção 2.2).

6.4 Complexidade

O Javadoc garante "log(n) time cost for the containsKey, get, put and remove operations" no TreeMap, e "log(n) time cost for the basic operations (add, remove and contains)" no TreeSet. Isso vale no pior caso, porque a altura da árvore rubro-negra é sempre O(log n), seja qual for a ordem de entrada.

Operação	TreeMap / TreeSet	Nossa AVL
inserir (put / add / adicionar)	O(log n)	O(log n)
buscar (get / contains / pesquisar)	O(log n)	O(log n)
remover (remove / remover)	O(log n)	O(log n)
percorrer em ordem	O(n)	O(n)

Tabela – ordem de complexidade no pior caso: TreeMap/TreeSet e a nossa AVL

Fontes: Javadoc do TreeMap (https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/TreeMap.html) e do TreeSet (https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/TreeSet.html), Java 21, e o código-fonte java/util/TreeMap.java do OpenJDK 21.